

Greedy

Greedy Method

미래를 내다보지 않고, 당장 눈 앞에 보이는 최적의 선택을 하는 방식

매 선택이 그 순간에서는 최적이지만 종합적으로 보았을 때 최적해임을 보장할 수는 없다. (물론 최적해일 경우도 있다.)

Greedy 알고리즘 예제

거스름돈 문제

당신이 편의점에서 일하는 점원이다. 카운터에는 500원, 100원, 50원, 10원짜리 동전이 무한히 존재한다고 하자. 손님에게 거슬러 줘야 할 돈이 N 원일 때 거슬러줘야 할 동전의 최소 개수를 구하여라. 단, 거슬러 줘야 할 돈 N 은 10의 배수이다.

당신이 편의점에서 일하는 점원이다. 카운터에는 500원, 100원, 50원, 10원짜리 동전이 무한히 존재한다고 하자. 손님에게 거슬러 줘야 할 돈이 N 원일 때 거슬러줘야 할 동전의 최소 개수를 구하여라. 단, 거슬러 줘야 할 돈 N 은 10의 배수이다.

[상황 정의]

거슬러 줄 돈 N 원, 동전 단위 $[C_1, C_2, C_3, C_4]$

(단, $C_1 > C_2 > C_3 > C_4$ 성립)

[그리디 로직]

- **1** 가장 큰 단위 C_1 부터 최대한 많이 채운다.
- **2** 남은 금액을 그다음 큰 단위 C_2 로 채운다.
- **3** 이 과정을 마지막 단위까지 반복한다.

당신이 편의점에서 일하는 점원이다. 카운터에는 500원, 100원, 50원, 10원짜리 동전이 무한히 존재한다고 하자. 손님에게 거슬러 줘야 할 돈이 N 원일 때 거슬러줘야 할 동전의 최소 개수를 구하여라. 단, 거슬러 줘야 할 돈 N 은 10의 배수이다.

[상황 정의]

거슬러 줄 돈 N 원, 동전 단위 $[c_1, c_2, c_3, c_4]$

(단, $c_1 > c_2 > c_3 > c_4$ 성립)

[그리디 로직]

- 1 가장 큰 단위 c_1 부터 최대한 많이 채운다.
- 2 남은 금액을 그다음 큰 단위 c_2 로 채운다.
- 3 이 과정을 마지막 단위까지 반복한다.

$n = 1260$

$cnt = 0$

큰 단위의 화폐부터 차례대로 확인

$coins = [500, 100, 50, 10]$

for coin in coins:

$cnt += n // coin$ # 해당 화폐로 거슬러 줄 수 있는 동전의 개수 세기

$n \% = coin$ # 거슬러 준 만큼 n 에서 빼주기

print(cnt)

그리디가 통하는 조건: 배수의 법칙

단위 간의 독립성

거스름돈 문제에서 그리디가 성립하려면, 큰 화폐 단위가 항상 작은 화폐 단위의 배수여야 합니다.

```
cnt += n // coin  
n %= coin
```

500, 100, 50, 10원은 서로 배수 관계이므로, 큰 것을 먼저 집는 선택이 이후의 선택을 망치지 않습니다.



탐욕법이 실패하는 합정

반례: [500, 400, 100]

800원을 거슬러 줘야 할 때:

Greedy: 500원(1) + 100원(3) = 총 4개

Optimal: 400원(2) = 총 2개

정당성 의심하기

'지금 내가 내린 선택이 다음 선택의 최적해를 방해하는가?'를 자문해야 합니다.

배수 관계가 깨지는 순간, 그리디는 정답을 보장할 수 없습니다.

실전 코딩테스트: 판별 4단계

STEP 1-2. 직관과 정렬

- 🔍 키워드 포착: '최소', '최대', '최단'을 요구하는가?
- 🔼 정렬(Sort) 연상: 정렬하면 답이 보일 것 같은가?

STEP 3-4. 검증과 확신

- 🕒 반례 찾기 (1분): 탐욕적인 규칙에 어긋나는 예외가 있는가?
- 🚫 독립성 확인: 한 번의 결정이 다음 선택지를 훼손하지 않는가?

반례가 떠오르지 않는다면 확신을 가지고 Greedy로 직진하세요!